Fix report: customers receive credits more than once

> **Sample deliverable (Starter, "Fix One Issue").** The client, "Acme Notes", is fictional and so is the code under repair. The evidence below is generated by `duplicate-credits-repro/repro.mjs`, not typed by hand.

Client	Acme Notes (fictional note-taking SaaS selling credit packs)
Issue	Paying customers occasionally see double (or more) credits after a single purchase
Scope	One handler: `invoice.paid`
Status	Fixed and reproduced/verified below; rollout needs the data-repair step in section 5

1. Symptom

Support tickets: "I bought one 500-credit pack and have 1000 credits", and one internal report where the balance shown in the app did not match the sum of the credit ledger.

2. Root cause

`invoice.paid` is handled by a function that adds credits **on every delivery** (`duplicate-credits-repro/handler.vulnerable.mjs`). Stripe delivers events **at least once**: retries after a slow or failed response, a manual resend, or a replay all deliver the same `event.id` again. Two defects combine:

1. **No idempotency.** Nothing records that an invoice was already credited, so each delivery credits again.
2. **Read-modify-write on the balance.** `balance = <value read earlier> + 500` loses updates when deliveries overlap, so under concurrency the balance and the ledger disagree.

3. Fix

One transaction; the ledger row is inserted first behind a **unique index on `stripe_invoice_id`** and the balance changes only if that insert won; the balance update is an atomic `balance = balance + n`.

```
--- handler.vulnerable.mjs
+++ handler.fixed.mjs
@@ -1,17 +1,28 @@
-// Deliberately insecure for demonstration. Do not deploy.
-// The flaw (seeded on purpose): credits are granted on EVERY delivery of invoice.paid.
-// Stripe delivers events at least once, so retries and replays double-credit the customer,
-// and the read-modify-write on the balance also loses updates under concurrency.
+// The fix: one transaction, idempotent on the invoice (and recorded per event), atomic balance update.
 export const CREDITS_PER_INVOICE = 500;

 export async function handleInvoicePaid(pool, schema, event) {
   const invoice = event.data.object;
-  const { rows } = await pool.query(`select id, balance from ${schema}.accounts where stripe_customer_id = $1`, [invoice.customer]);
-  if (!rows[0]) throw new Error("unknown customer");
-  const account = rows[0];
-  await pool.query(`update ${schema}.accounts set balance = $1 where id = $2`, [account.balance + CREDITS_PER_INVOICE, account.id]);
-  await pool.query(
-    `insert into ${schema}.credit_ledger (account_id, amount, stripe_event_id, stripe_invoice_id) values ($1,$2,$3,$4)`,
-    [account.id, CREDITS_PER_INVOICE, event.id, invoice.id],
-  );
+  const client = await pool.connect();
+  try {
+    await client.query("BEGIN");
+    const acct = await client.query(`select id from ${schema}.accounts where stripe_customer_id = $1`, [invoice.customer]);
+    if (!acct.rows[0]) throw new Error("unknown customer");
+    // The unique index on stripe_invoice_id is the guard: a replay or a second event for the same invoice inserts nothing.
+    const ins = await client.query(
+      `insert into ${schema}.credit_ledger (account_id, amount, stripe_event_id, stripe_invoice_id)
+       values ($1,$2,$3,$4) on conflict (stripe_invoice_id) do nothing returning id`,
+      [acct.rows[0].id, CREDITS_PER_INVOICE, event.id, invoice.id],
+    );
+    if (ins.rowCount === 1) {
+      // Atomic increment, not read-modify-write.
+      await client.query(`update ${schema}.accounts set balance = balance + $1 where id = $2`, [CREDITS_PER_INVOICE, acct.rows[0].id]);
+    }
+    await client.query("COMMIT");
```

```
+  } catch (e) {
+    await client.query("ROLLBACK").catch(() => {});
+    throw e;
+  } finally {
+    client.release();
+  }
 }
```

4. Test added

`duplicate-credits-repro/repro.test.mjs` replays the same `invoice.paid` event several times, sequentially and concurrently, against both handlers. It asserts the vulnerable handler over-credits (this documents the seeded flaw) and the fixed one credits exactly once. It is wired into CI against Postgres and passes both in CI and locally.

5. Verification

What was run for this sample: the repro above against a local Postgres, replaying one synthetic `invoice.paid` event. Results (generated):

One event delivered 5 times; each paid invoice is worth 500 credits.

Handler / delivery	Deliveries	Ledger rows	Balance	Sum of ledger
vulnerable, sequential	5	5	2500	2500
vulnerable, concurrent	5	5	1000	2500
fixed, sequential	5	1	500	500
fixed, concurrent	5	1	500	500

The vulnerable concurrent row shows the second symptom: the ledger and the balance disagree.

How you verify in your own Stripe test mode (a real engagement runs these; they are not claimed as run here):

1. Point the Stripe CLI at your dev endpoint: `stripe listen --forward-to localhost:3000/api/webhooks/stripe`.
2. Trigger a paid invoice: `stripe trigger invoice.paid`, note the `evt_...` id.
3. Deliver it again by replaying the captured payload with a valid signature (or `stripe events resend evt_... --webhook-endpoint=we_...` if you have a registered endpoint).

4. Expect one ledger row, one credit, and a duplicate logged. Repeat with several concurrent deliveries.

Live evidence for the same mechanism (idempotent insert keyed on `event.id`, recomputed state, real Stripe events) is in `docs/chaos.md`.

Roll out in this order: repair, index, then deploy. The unique index cannot be created while duplicate rows exist, and the fixed handler's `ON CONFLICT (stripe_invoice_id)` fails with "no unique or exclusion constraint matching the ON CONFLICT specification" until the index exists, so deploying first would break every `invoice.paid`. (a) Run a one-off script that, per invoice, keeps the first ledger row, reverses the extra credits from balances, and records the correction in an audit table; review the affected accounts with the client before reversing anything. (b) Create the index (`CREATE UNIQUE INDEX CONCURRENTLY`, so writes are not blocked). (c) Deploy the fixed handler. The repro clears its scratch tables before adding the index; it does not exercise this migration.

6. Follow-ups (not included in this fix)

- Apply the same idempotency to any other event that grants something (`checkout.session.completed`, `customer.subscription.*`).
- Store `event.id` in a processed-events table and process from a queue so the endpoint returns quickly (see the kit's webhook rules).
- Add a reconciliation job so a missed webhook is repaired instead of silently losing credits.